

TP — Browser Fingerprinting

Imane Fouad | UM6P

Mission: Follow Alice

Previously, in The Alice Files...

You successfully (I hope...) tracked Alice using cookies stored in her browser. But Alice discovered your technique. She deleted every cookie, activated cookie blocking and confidently closed her browser.

“That should stop them,” she thought.

Plot twist: cookies are not the only way to recognize a browser.

Every browser reveals small clues about its environment: screen properties, language, time zone, graphics rendering and hardware capabilities. Individually, these clues may appear harmless. Together, they may form a recognizable fingerprint.

Your new mission is to collect these clues, build a fingerprinting prototype and determine whether Alice can still be recognized without storing an identifier in her browser.

Mission : Find Alice without cookies.

You can work as both:

- a **privacy researcher**, implementing a small fingerprint;
- a **defender**, testing its limits (Optional; feel free to try different protection tools and compare the results).

Investigator's Note

This TP must remain local and transparent. Test only browsers and machines for which you have permission.

Objectives

By the end of the session, you should be able to:

- identify information exposed through HTTP requests;
- collect additional browser and device features using JavaScript;
- distinguish passive, active and behavioural fingerprinting;
- build a simple typing-behaviour measurement prototype;
- combine several features into a browser fingerprint;

1 Part 1 — Investigate the HTTP Request

1.1 Mission: Discover Alice's Passive Clues

Alice has blocked cookies, but she still needs to communicate with your web server. Every time she requests a page, her browser automatically sends an HTTP request containing information about its environment.

This information is collected **passively**: the server receives it without running JavaScript and without asking the browser to store anything.

What can the server learn from a single HTTP request?

1.2 Set Up the Project

Create the following project structure:

```
fingerprintingLab/  
|-- app.py  
|-- templates/  
|   |-- index.html  
|-- static/  
|   |-- fingerprint.js
```

The file `fingerprint.js` will be used later for active fingerprinting. During this first part, you must collect information **without executing JavaScript**.

Activate the Python environment used in the previous lab, or create a new one, then install Flask:

```
python -m venv venv  
venv\Scripts\activate  
pip install flask
```

1.3 Learn about the passive features

Before the browser displays a webpage, it sends an HTTP request to the server. This request contains headers that may reveal information about the browser. First, investigate these header on the client side Open Developer Tools and select:

Network -> Reload the page -> Select the request -> Headers

Task0: Investigate the HTTP Request

Research the different HTTP headers, focusing primarily on those sent in the **HTTP request**. Determine what information each header reveals and which features could contribute to browser fingerprinting.

1.4 Collect the passive feature

Now, you need to collect these feature on the server side. Update `app.py` as follows:

```
from flask import Flask, render_template, request  
  
app = Flask(__name__)
```

```

@app.route("/")
def home():
    print("\n===== NEW VISIT =====")

    # Information about the request and network connection
    print("IP address      :", request.remote_addr)
    print("HTTP method    :", request.method)

    # Examples of passive HTTP features
    print("User-Agent      :", request.headers.get("User-Agent"))
    print("Accept-Language :", request.headers.get("Accept-Language"))

    # TODO: Add the other passive features identified in Task 0.
    # Example:
    # print("Feature name   :", request.headers.get("Header-Name"))

    print("=====\n")

    return render_template("index.html")

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)

```

Task 1: Expand the Passive Collector

Investigate what other features a website can collect passively from an HTTP request. Use reliable documentation and search for information related to:

- browser and operating-system information;
- preferred content formats and languages;
- navigation context;
- security-related headers.

Then:

1. identify at least **four additional HTTP headers**;
2. update `app.py` to print their values;
3. reload the website and record the observed values;
4. test the page using another browser or private-browsing mode;
5. identify which values remain the same and which values change.

2 Part 2 — Active Feature Collection

2.1 From Passive to Active Fingerprinting

In the previous part, the server collected information that the browser sent automatically with the HTTP request. This is called **passive fingerprinting**.

Active fingerprinting occurs when a website executes code in the browser to query browser APIs and collect additional features.

For example, JavaScript can ask the browser about:

- its language and time zone;
- the screen resolution and colour depth;
- the number of logical processor cores;
- the size of the browser window;
- its graphics and rendering capabilities.

The browser does not necessarily include these values in the original HTTP request. The website must actively retrieve them using JavaScript.

Server sends JavaScript → Browser executes it → JavaScript reads browser features

Some browser properties can be read without requesting permission. More sensitive APIs, such as precise geolocation, camera and microphone access, normally require the user's permission.

You can refer to the Appendix to read more about JavaScript

2.2 Add JavaScript to the Webpage

Your project already contains the following JavaScript file:

`static/fingerprint.js`

Open `templates/index.html` and add the following code before the closing `</body>` tag:

```
<h2>Active Feature Collection</h2>

<p id="feature-output">
  Waiting for JavaScript...
</p>

<script src="{ { url_for('static',
                        filename='fingerprint.js') }}"></script>
```

2.3 Read Your First Active Feature

Add the following code to `static/fingerprint.js`:

```
// Ask the navigator object for the browser's preferred language
const browserLanguage = navigator.language;

// Display the collected value in the Developer Tools console
console.log("Browser language:", browserLanguage);

// Find the HTML element whose identifier is "feature-output"
const outputElement =
  document.getElementById("feature-output");

// Display the collected value inside the webpage
outputElement.textContent =
  "Browser language: " + browserLanguage;
```

Task 2 — Expand the Active Collector

Consult the table provided in the Appendix. Identify additional active features that could contribute to browser fingerprinting.

Update your JavaScript code to:

1. collect at least 4 additional relevant features from different browser objects;
2. display the collected values on the webpage;
3. send the collected features to the Flask server;
4. print the received features in the Flask console.

For each selected feature, briefly explain what it reveals and why it could help distinguish one browser from another.

5. Repeat the experiment using different protection mechanisms, such as private-browsing mode, cookie blocking, and privacy preserving extensions. Identify which features change or remain stable, then determine which protection is most effective against browser fingerprinting and explain its limitations.

3 Part 3 — The typing imposter

Browser fingerprinting is not limited to device and browser properties. A website may also observe **how a user interacts** with its pages.

In real systems, typing behaviour may be collected from many input fields and over multiple sessions. A large number of keyboard events can be combined to identify behavioural patterns such as:

Feature	Meaning
Typing speed	The number of characters or words typed during a period of time.
Key-hold time	The time between pressing a key and releasing it.
Inter-key interval	The delay between two consecutive key presses.
Pauses	Long periods during which the user stops typing.
Corrections	The number and frequency of Backspace or Delete presses.

Collecting many samples may produce a behavioural profile. However, the result is probabilistic: typing behaviour can change because of the keyboard, fatigue, stress, posture or the user's environment.

3.1 Your Prototype

To keep the experiment simple, Alice will not type naturally across several webpages. Instead, she will be asked to reproduce one fixed sentence:

Cookies are gone, but Alice's rhythm still leaves traces!

In this prototype, focus on only two behavioural characteristics:

- Alice's **typing speed**;
- the number of **corrections** made using Backspace.

Alice must type the sentence exactly. The timer begins when she types her first character and stops when her input matches the target sentence.

Task 3

1. the sentence that Alice must reproduce;
2. a text input in which Alice can type;
3. a button for restarting the experiment;
4. an initially empty area for displaying:
 - the total typing time;
 - the typing speed;
 - the number of corrections.

Give each element a meaningful and unique `id` so that it can later be accessed from JavaScript.

Task 2 — Build the JavaScript Collector

The JavaScript program must observe Alice while she reproduces the target sentence. It should start a timer when she types her first character, count her corrections and stop when her input exactly matches the displayed sentence.

The program must then calculate Alice's typing speed, display the results on the webpage and send the resulting behavioural features to the Flask server.

Include your JavaScript file at `index.html`:

Task 4 — Alice's Typing Rhythm

Your task is to:

1. define the formula for calculating typing speed;
2. consult the JavaScript reference table in the Appendix;
3. design a short algorithm before writing the code;
4. create `static/fingerprint2.js` so that it:
 - starts timing with Alice's first character;
 - counts her Backspace corrections;
 - detects when the sentence is reproduced exactly;
 - calculates the total time and typing speed;
 - displays the results on the webpage;
5. send the typing time, typing speed and correction count to the Flask server;
6. verify the request in Developer Tools and the received data in the Flask console.

4 Part 4 — Give the Clues One Identifier

The information collected about Alice currently consists of several separate features: language, screen size, time zone, hardware properties and typing behaviour.

A tracker can combine these features and apply a **hash function** to produce one fixed-length identifier:

Language + Screen + Time zone + Behaviour $\xrightarrow{\text{Hash}}$ Fingerprint identifier

For example:

```
en-US | 1920x1080 | Africa/Casablanca | 8 | 5.2
      |
      v
8b91c7a4...e27a
```

The hash is useful because:

- the same combination produces the same identifier;
- it is shorter and easier for the server to compare;
- the raw features are no longer immediately readable in the request;

A request may therefore contain:

```
/collect?fp=8b91c7a4...e27a
```

Alice may see only an apparently meaningless value instead of the collected features. This helps explain why fingerprinting can be difficult for ordinary users to notice.

Optional Mission — Build the Fingerprint

Using the SHA-256 hashing function :

1. combine the features collected during the previous challenges in a fixed order;
2. convert the combined features into one string;
3. investigate JavaScript's `crypto.subtle.digest()` function;
4. use **SHA-256** as the hashing algorithm;
5. display the resulting fingerprint on the webpage;
6. generate it several times (excluding the behavioral fingerprints) and test whether it remains stable;
7. Compare your fingerprint with your colleagues;
8. send only the hash to the server.

Appendix

A—A Short Introduction to JavaScript

JavaScript is a programming language executed inside the browser. It can interact with the webpage and with objects exposed by the browser.

A property is usually accessed using the following notation:

```
object.property
```

For example:

```
navigator.language
```

Here, `navigator` is the browser object and `language` is one of its properties.

4.1 Useful Browser Objects

Object	What it represents	Examples of accessible features
<code>navigator</code>	Information about the browser and device environment.	<code>language</code> , <code>languages</code> , <code>userAgent</code> , <code>hardwareConcurrency</code> , <code>deviceMemory</code>
<code>screen</code>	Information exposed about the user's screen.	<code>width</code> , <code>height</code> , <code>availWidth</code> , <code>colorDepth</code>
<code>window</code>	The current browser window and its display environment.	<code>innerWidth</code> , <code>innerHeight</code> , <code>devicePixelRatio</code>
<code>Intl</code>	The browser's international and regional configuration.	Time zone, locale and formatting preferences
<code>document</code>	The current webpage and its navigation context.	<code>referrer</code> , <code>title</code> , <code>visibilityState</code>

These values may be unavailable, reduced or modified by the browser's privacy protections. A collected value should therefore never be treated as guaranteed proof of a person's identity.

B – JavaScript Survival Guide

You do not need to master JavaScript to complete this challenge. Your script only needs to:

1. access elements from the HTML page;
2. observe Alice's keyboard activity;
3. remember a few values;
4. perform a calculation;
5. display and send the result.

Essential JavaScript Elements

What you need	JavaScript	Purpose
Create a fixed variable	<code>const name = value;</code>	Store a value that will not be re-assigned.
Create a variable that changes	<code>let count = 0;</code>	Store a timer, counter or experiment state.
Access an HTML element	<code>document.getElementById("id")</code>	Find an element using its HTML identifier.
Read an input	<code>input.value</code>	Obtain the text currently entered by Alice.
Read displayed text	<code>element.textContent</code>	Obtain the sentence contained in an element.
Display a result	<code>element.textContent = value;</code>	Modify the text shown on the web-page.
Count characters	<code>text.length</code>	Obtain the number of characters in a sentence.
Observe an action	<code>element.addEventListener(...)</code>	Execute instructions when an event occurs.
Identify a key	<code>event.key</code>	Determine whether Alice pressed Backspace or another key.
Measure time	<code>performance.now()</code>	Obtain a timestamp in milliseconds.
Test a condition	<code>if (condition) {...}</code>	Execute code only when a condition is true.
Compare two values	<code>value1 === value2</code>	Test whether two values are equal.
Format a number	<code>number.toFixed(2)</code>	Keep two digits after the decimal point.
Convert data to JSON	<code>JSON.stringify(data)</code>	Prepare a JavaScript object for transmission.
Send an HTTP request	<code>fetch("/collect", ...)</code>	Send the calculated features to Flask.

The Three Events You Need

Event	When it occurs
<code>keydown</code>	When Alice presses a key. Use it to start the timer and count Backspace presses.
<code>input</code>	When the content of the input field changes. Use it to check whether Alice has completed the sentence.
<code>click</code>	When Alice selects a button. Use it to restart the experiment.

The general event-listener structure is:

```
element.addEventListener("event-name", function (event) {
    // Instructions executed when the event occurs
});
```

For example, to detect a key:

```
input.addEventListener("keydown", function (event) {
    console.log(event.key);
});
```

Recommended Construction Order

Build your script progressively:

1. use `getElementById()` to access the sentence, input and result area;
2. create variables for the starting time and correction count;
3. use `keydown` to start timing and detect Backspace;
4. use `input` to detect when the two sentences match;
5. calculate and display the results;
6. place the results in an object and send it to `/collect`.